Understanding class definitions 2

Exploring source code

- Produced by: Ms. Mairead Meagher, Ms. Siobhán Roche.

Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University

Department of Computing and Mathematics
http://www.setu.ie/

Upcoming

- Methods, including:
  - *accessor* (*getter*) methods
  - *mutator* (*setter*) methods;
- String formatting;
- Conditional statements;
- Local variables. (next slidedeck)

Methods

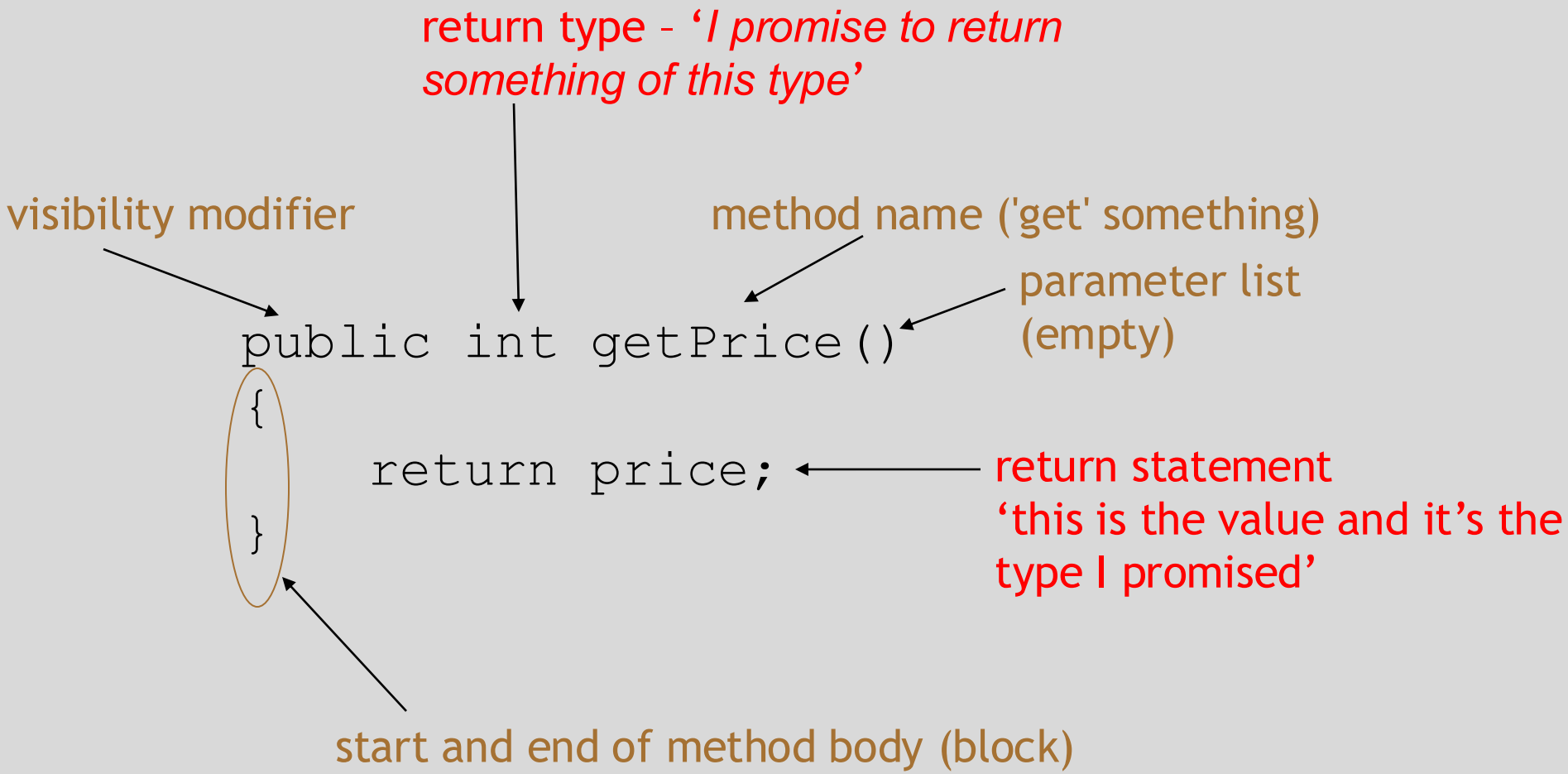
- Methods implement the *behaviour* of objects.
- Methods have a consistent structure comprised of a *header* and a *body*.
- *Accessor methods* provide information about an object.
- *Mutator methods* change the state of an object.
- Other sorts of methods accomplish a variety of tasks.

Method structure

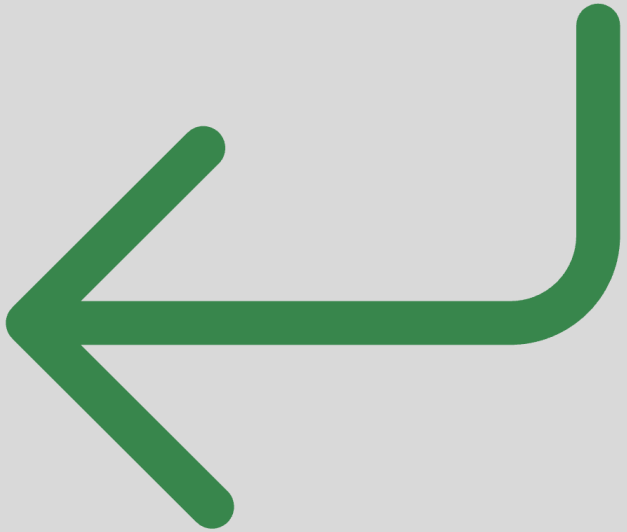
- Accessor methods often occur in the form of *getter* methods.
- The header:
  - **`public int getPrice()`**
- The header tells us:
  - the *visibility* to objects of other classes (public);
  - whether the method *returns a result* (an integer);
  - the *name* of the method *(*getPrice*);*
  - whether the method takes *parameters* (none, in this case).

- The body encloses the method's *statements*.

Getter methods



Accessor methods



- An accessor method always has a return type that is not void.
- An accessor method returns a value (result) of the type given in the header.
- The method will contain a return statement to return the value.
- NB: Returning is not printing!
- It presents a value to the calling method.

Test – syntax errors

```
public class CokeMachine
{
    private price;

    public CokeMachine()
    {
       price = 300
    }

    public int getPrice
    {
       return Price;
    }
```

- What is wrong here? There are five errors.

Test

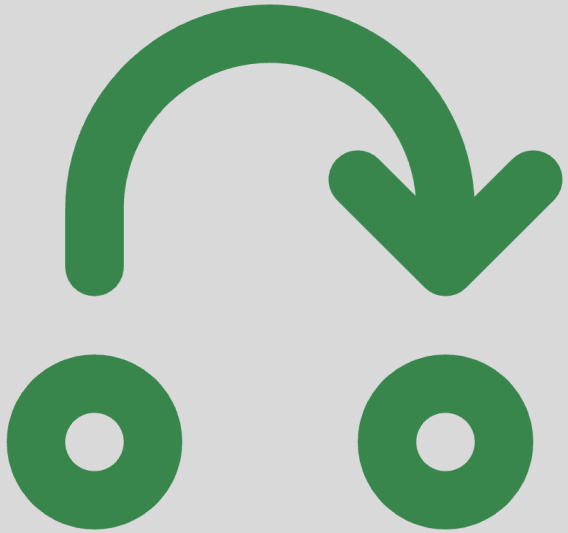
```
public class CokeMachine
{
        int
    private price;

    public CokeMachine()
    {
        price = 300 ;
    }

    public int getPrice ()
    {
        return Price;
    }
}
```

- What is wrong here? There are five errors.

Mutator methods



- Have a similar method structure: header and body.
- Used to *mutate* (i.e., change) an object's state.
- Achieved through changing the value of one or more fields.
  - They typically contain one or more assignment statements.
  - Often receive parameters.
- The most basic mutator methods are 'setter' methods.

setter methods

- Fields often have dedicated **`set`** mutator methods.

- These have a simple, distinctive form:
  - **`void`** return type
  - method name related to the field name
  - single formal parameter, with the same type as the type of the field
  - a single assignment statement

A typical **`set`** method

```
public void setDiscount(int amount)
{
    discount = amount;
}
```

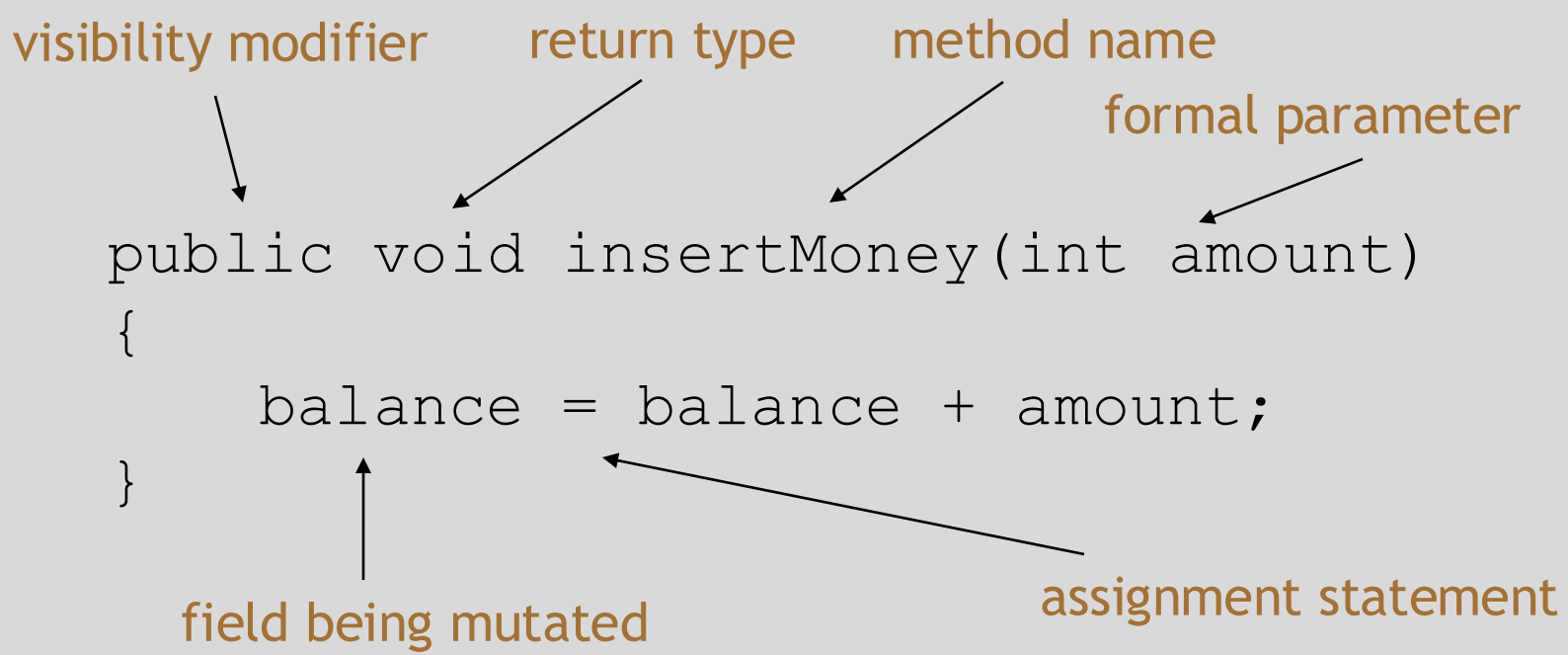
We can easily infer that **`discount`** is a field of type **`int`**, i.e:

```
private int discount;
```

Protective mutators

- A set method does not have to always assign unconditionally to the field.
- The parameter may be checked for validity and rejected if inappropriate.
  - e.g. only update field if new value is >=10
- Mutators thereby protect fields.
- Mutators support *encapsulation*.

Another mutator method



This method *changes* the value of balance; hence the object's state is *mutated*.

The `this` keyword

- Up to now we have come up with a different name for the parameter and the field.
- However, sometimes there is one name that perfectly describes the use of a variable—it fits so well that we do not want to invent a different name for it.

- **`Enter the this. construct`**

The `this` keyword

- Used to distinguish parameters and fields of the same name.
  E.g.:

```
public TicketMachine(int price)
{
    this.price = price;
    balance = 0;
    total = 0;
}
```

The `this` keyword

- Used to distinguish parameters and fields of the same name. E.g.:

```
public class TicketMachine
{
    private int price;
    private int balance;
    private int total;


public TicketMachine(int price)
{
    this.price = price;
    balance = 0;
    total = 0;
}
}
```

The `this` keyword

```
this.price = price;
```

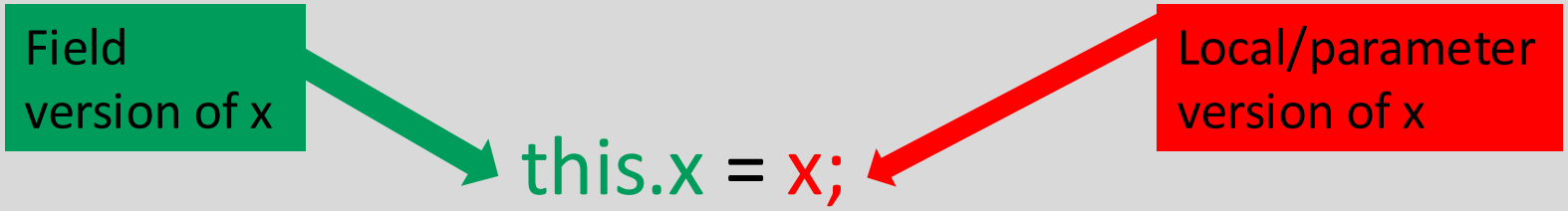
This statement has the following effect:

field named price = *parameter named* price;

The **this** keyword

- The **this** keyboard can **always** be used when referencing field variables.

- In practice, we use it when there is a parameter with the same name as a field.



Printing from methods

```
public void printTicket()
{
    // Simulate the printing of a ticket.
    System.out.println("##################");
    System.out.println("# The BlueJ Line");
    System.out.println("# Ticket");
    System.out.println("# " + price + " cents.");
    System.out.println("##################");
    System.out.println();

    // Update the total collected with the balance.
    total = total + balance;
    // Clear the balance.
    balance = 0;
}
```

String concatenation

- 4 + 5

  9
- "wind" + "ow"

  "window"
- "Result: " + 6

  "Result: 6"
- "# " + price + " cents"

  "# 500 cents"

Quiz

- System.out.println(5 + 6 + "hello");

- System.out.println("hello" + 5 + 6);

Quiz

- System.out.println(5 + 6 + "hello");
- System.out.println("hello" + 5 + 6);

11hello

hello56

Formatted printing

- Concatenation can be used to create output in a desired format.
- An alternative is to use **`printf`**.
- The first argument gives the overall structure.
- Format specifiers represent 'holes' to be filled in by the other arguments.

Formatted printing

- Concatenation:
```
System.out.println("# " + price + "
cents.");
```
- Using **`printf`**:
```
System.out.printf("# %d cents.%n",
price);
```
- **`%d`** means insert the next argument as an integer value at this point.
- **`%n`** means 'end the line' (i.e., 'insert' a newline) at this point.

Method summary

- Methods implement all object behavior.
- A method has a name and a return type.
  - The return-type may be **`void`**.
  - A non-**`void`** return type means the method will return a value to its caller.
- A method might take parameters.
  - Parameters bring values in from outside for the method to use.

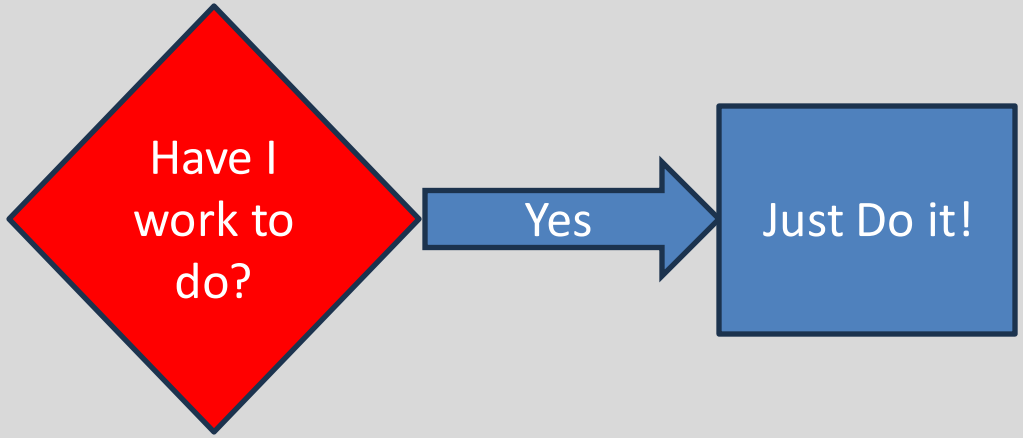
Reflecting on the ticket machines

- Their behavior is inadequate in several ways:
  - No checks on the amounts entered.
  - No refunds.
  - No checks for a sensible initialization.
- How can we do better?
  - We need the ability to choose between different courses of action.

Conditionals – the if statement

Making choices in everyday life

- If I have an assignment to complete, then I shall work on my assignment



Conditional Statement Syntax (1)

```
if(condition1…perform some test)
{
     Do these statements if the test gave a true result
}
```

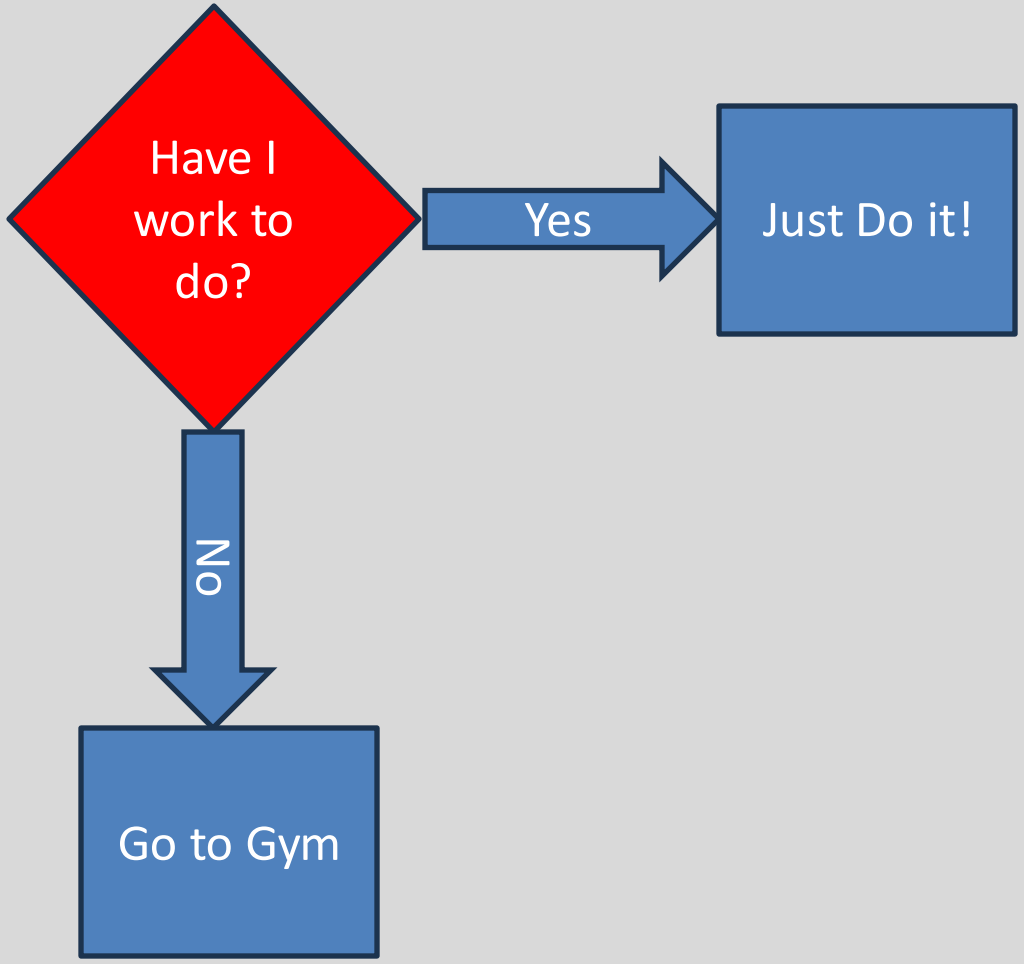
Making a choice in the ticket machine (1)

```
public void insertMoney(int amount)
{
    if(amount > 0) {
        balance = balance + amount;
    }
}
```

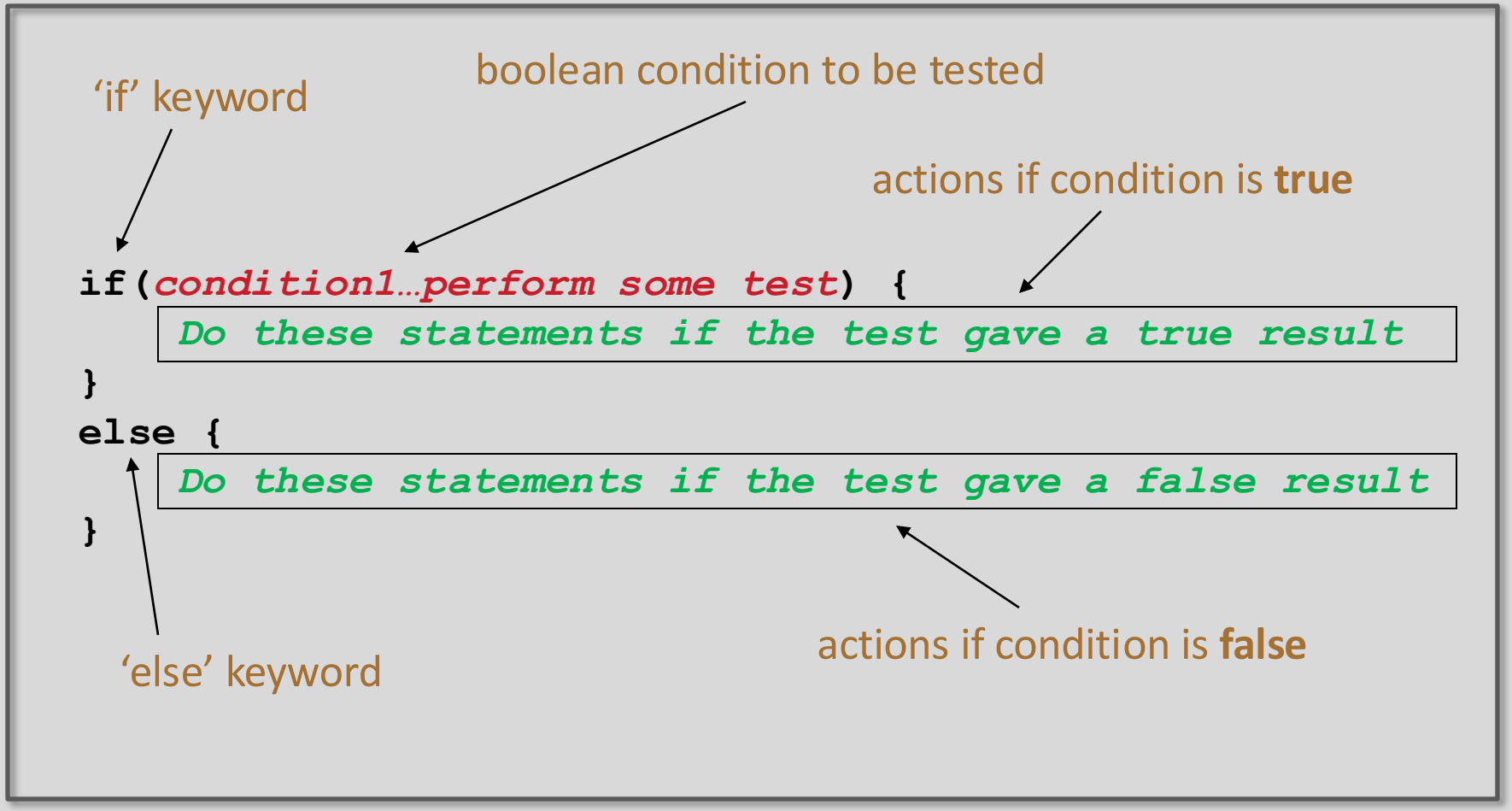
the conditional statement avoids an inappropriate action

Making choices in everyday life (2)

- If I have an assignment to complete, then I shall work on my assignment
- Otherwise I will go to the Gym



Conditional Statement Syntax (2)



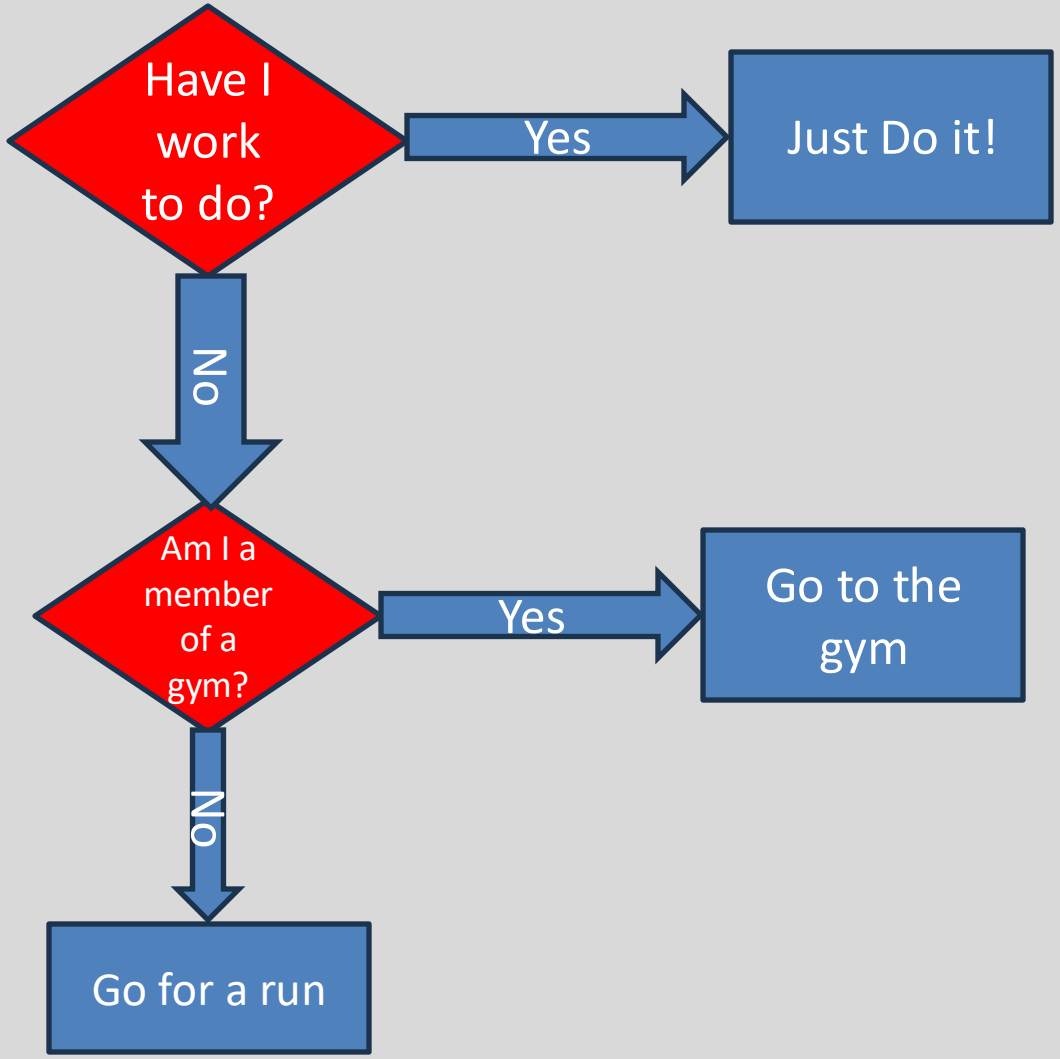
Making a choice in the ticket machine (2)

```
public void insertMoney(int amount)
{
    if(amount > 0) {
        balance = balance + amount;
    }
    else {
        System.out.printf(
           "Use a positive amount: %d%n",
            amount);
    }
}
```

the conditional statement avoids an inappropriate action

Making choices in everyday life (3)

- If I have an assignment to complete, then I shall work on my assignment
- Otherwise if I am a member of a gym, go to the gym
- Otherwise I will go for a run



Conditional Statement Syntax (3)

```
if(condition1…perform some test)
{
    Do these statements if condition1 gave a true result
}
else if(condition2…perform some test)
{
    Do these statements if condition1 gave a false
    result and condition2 gave a true result
}
else
{
    Do these statements if both condition1 and
    condition2 gave a false result
}
```

Making a choice in the ticket machine (3)

```
public void specialOffer(int amount)
{
    if(amount >100) {
        balance = balance + 50;
    }
    else if amount > 50{
        balance = balance + 25;
    }
    else amount = amount + 5;
}
```

Note that if any condition is true, the associated statements are executed AND the if statement is finished.

Boolean conditions

- A boolean condition is an expression that evaluates to either **true** or **false** e.g.

  price < 50

- An if statement evaluates a **boolean condition** and its result will determine which portion of the if statement is executed.

Boolean conditions

```
// Do these statements before.

if (boolean condition)
{
    // Perform this clause if the
        // condition is true.
}

// Do these statements after.
```

Java Relational Operators

Operator	Use	Returns true if…
>	op1 > op2	op1 is **greater** than op2
>=	op1 >= op2	op1 is **greater than or equal** to op2
<	op1 < op2	op1 is **less** than to op2
<=	op1 <= op2	op1 is **less than or equal** to op2
==	op1 == op2	op1 and op2 are **equal**
!=	op1 != op2	op1 and op2 are **not equal**

BEWARE = is an assignment operator.
It doesn't test for equality. Use == to test for equality in primitive types

Source: http://www.freejavaguide.com/relational_operators.htm

Some notes on the if statement

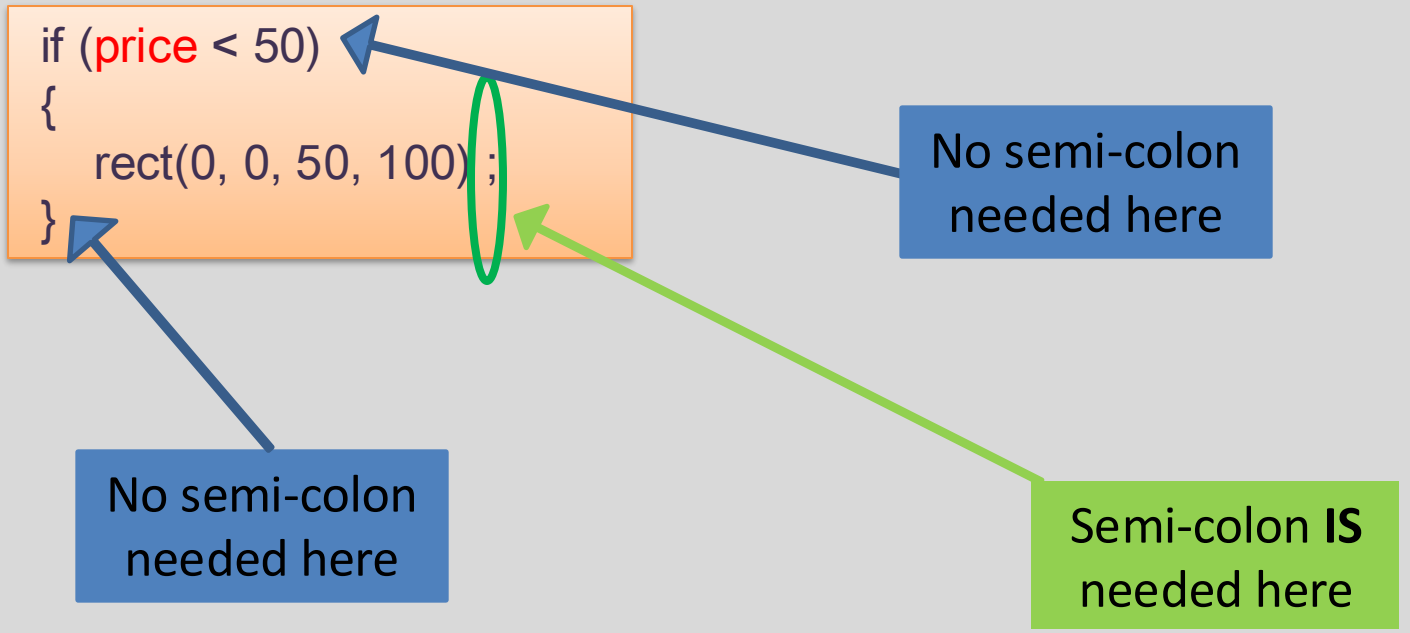
- An if statement IS a **statement**;
  it is only executed once.
- When your if statement only has one statement inside it, you do not need to use the curly braces.
- For example, both of these are the same:

```
if (price < 50)
{
   price = 50;
}
```

```
if (price < 50)
   price = 50;
```

Some notes on the if statement

- The semi-colon (;) is a **statement terminator**.



Questions?